swingtradev3

Phase 1 — Full System Design Document v6.0

Swing Trading Autonomous Agent · Nifty 200 · Zerodha Kite · NVIDIA NIM · GTT Exits · Human-in-Loop

---

Version **5.0** · March 2026 · v6: fundamentals layering, GTT corporate action logic, model-agnostic config, SKILL.md versioning, sector concentration, async scan, entry validity

Style	Universe	Hold period	Capital
Swing trade	Nifty 200	7–14 days	₹20,000

Agents	Exit method	Max positions	Monthly cost
Two only	Zerodha GTT	3 concurrent	₹530

1. System Philosophy

swingtradev3 mimics the decision-making of a professional swing trader using NIM as the analytical brain. It does not run hardcoded strategies. It reads a trading philosophy document (SKILL.md) and evaluates each stock through that lens — exactly how a seasoned trader internalises their approach and applies it to every setup they see.

1.1 The autoresearch pattern — adapted for trading

Inspired by Karpathy's autoresearch: an autonomous research loop built on three primitives. The code is plumbing. The markdown files are the intelligence.

Primitive	autoresearch original	swingtradev3 adaptation
Editable asset	train.py — the ML training script the agent modifies	strategy/SKILL.md — the trading philosophy the agent proposes changes to. Updated monthly after sufficient trade evidence. Human approves every change.
Scalar metric	val_bpb — validation loss. Lower = better.	Rolling Sharpe on last 20 closed trades. Per-stock: NIM confidence score 0–10. Setups scoring below 7.0 are discarded.
program.md	Instructions + constraints + stopping criteria for the agent.	strategy/research_program.md — the research procedure. How to fetch data, what questions to ask, how to score, what disqualifies. Changed rarely.

1.2 Why the analyst loop is monthly, not weekly

With a 7–14 day average hold and maximum 3 positions, you close roughly 4–8 trades per month. Weekly review means you are analysing 1–2 trades at a time — statistical noise, not a pattern. Proposing SKILL.md changes from 2 trades is guaranteed overfitting.

Loop	Trigger	Min trades	Purpose
Trade review	Event-driven — fires when each trade closes	N/A	Logs structured notes per trade: did thesis hold? what triggered exit? No SKILL.md changes. Just observation.
Monthly pattern analysis	Monthly — first Sunday of the month	8 trades	Analyst agent reviews all closed trades. Finds patterns in what worked and failed. Proposes specific SKILL.md edits with supporting evidence.
Quarterly strategy audit	Quarterly — every 3 months	20 trades	Is SKILL.md still coherent after monthly edits? Run fresh backtest. Check WFE

			ratio still above 0.5. Validate thresholds.

1.3 One codebase, three modes — the flag-driven architecture

The same research logic, same indicators, same NIM calls, same risk gate run in all three modes. The trading.mode flag in config.yaml controls only two things: where data comes from and where orders go. There are no mock classes, no test doubles, no monkey-patching.

trading.mode	Data source	Order destination	GTT handling
backtest	kite_fetcher.py calls kite.historical_data() in chunks	paper/fill_engine.py — next-day-open + slippage + brokerage	paper/gtt_simulator.py — checks stop/target hit per daily candle
paper	kite_fetcher.py calls live EOD data from Kite	paper/fill_engine.py — same as backtest, real-time LTP	paper/gtt_simulator.py — tracks trigger prices, no real Kite GTT
live	kite_fetcher.py calls live EOD data from Kite	tools/order_execution.py → kite.place_order() — real money	tools/gtt_manager.py → kite.place_gtt() — real Zerodha GTT

How the flag works — no mocks, no conditionals in business logic

kite_fetcher.py reads cfg.trading.mode at startup. If backtest: chunks historical data and returns it as a DataFrame. If paper|live: fetches latest EOD. The caller (research_agent, execution_agent) gets a DataFrame either way — same format, same columns, no difference. order_execution.py reads the flag before placing: if paper|backtest, calls paper/fill_engine instead of kite.place_order(). gtt_manager.py does the same. The flag is read once per call — not scattered through business logic.

2. Architecture

Two agents. Everything else is a module they call. All intelligence lives in the markdown files. All configuration lives in config.yaml. All secrets live in .env.

2.1 The two agents

Agent	Schedule	Core responsibility
research_agent.py	Every evening 15:45 → morning briefing 08:45	Scans all 200 Nifty 200 stocks. Runs quick Python filter. Calls NIM with SKILL.md + research_program.md for each surviving stock. Scores 0–10. Shortlists top 7. Runs monthly SKILL.md improvement loop.
execution_agent.py	Polls every 30 min, 09:15–15:30	Receives your YES/NO approval. Places limit order on YES. Immediately places two GTTs (stop-loss + target). Monitors positions daily. Trails stops when trade moves in favour. Alerts on thesis changes.

2.2 The three strategy files

File	Purpose	Updated by	Update frequency
strategy/SKILL.md	The trading philosophy. What makes a great swing setup. How to think about entries, stops, catalysts. Written in prose NIM can parse and apply.	You + monthly analyst loop	Monthly min
strategy/research_program.md	The research procedure. Step-by-step: fetch this data, ask NIM these questions, score like this, disqualify on these conditions. The process, not the philosophy.	You only	Rarely
strategy/analyst_program.md	The monthly review procedure. How to analyse closed trades, what patterns to look for in SKILL.md failures, how to write specific edits with supporting evidence.	You only	Rarely

2.3 Complete order lifecycle

Step	Time	Actor	Action

1	Evening	research_agent	Scans Nifty 200. NIM scores each stock via SKILL.md. Shortlist written to context/pending_approvals.json.
2	08:45	research_agent	Telegram morning briefing: each shortlisted stock with score, setup type, entry zone, stop, target, 2-sentence thesis. "Reply YES/NO for each."
3	Morning	You	Reply YES or NO to each setup. One tap. This is your only required daily action.
4	09:15+	execution_agent	check_risk() → if approved: place_order() limit entry → wait for fill confirmation.
5	On fill	execution_agent	Immediately place two GTT orders: (a) SL-M stop at stop_price, (b) Limit sell at target_price. Both on Zerodha servers. Bot can now be offline — exits are safe.
6	Every 30 min	execution_agent	Poll kite.positions() + kite.get_gtts(). Did any GTT trigger? Is stop still active? Update state.json. Alert if anything changed.
7	If +5% move	execution_agent	modify_gtt() stop to breakeven. If +10%: trail stop to entry+5%. Only tightens — never widens.
8	Evening	research_agent	Re-scores all open positions nightly. If thesis score drops below 5.0: Telegram alert "thesis weakened — consider exit." Never exits autonomously.
9	On close	execution_agent	GTT triggers (stop or target hit) → update state.json → log full trade to trades.json with NIM reasoning → alert you → event-driven trade review logs observation.

3. Full Directory Structure

```
swingtradev3/
│
├── main.py                              # Entry — schedules research_agent + execution_agent
├── config.py                            # Loads config.yaml → typed Pydantic Settings
├── config.yaml                          # ALL tunable values — committed to git, no secrets
├── requirements.txt
├── .env                                 # Secrets only — API keys. Never commit.
├── .env.example                         # Template — commit this
├── PAUSE                                # Kill switch — touch to halt execution_agent
immediately
├── .gitignore                           # Includes: .env, .backtest_cache/, reports/,
__pycache__/
├── README.md
│
├── agents/
│   ├── __init__.py
│   ├── research_agent.py                # Evening scanner + morning briefing + monthly
analyst loop
│   ├── execution_agent.py               # Entry approval + GTT lifecycle + 30-min position
polls
│   └── reconciler.py                    # Startup: state.json vs live Kite positions
│
├── llm/
│   ├── __init__.py
│   ├── nim_client.py                    # NVIDIA NIM via OpenAI-compatible SDK
│   ├── router.py                        # Fallback: NIM (5s timeout) → Groq → Gemini →
Claude
│   ├── prompt_builder.py                # SKILL.md + research_program.md + stock data →
system prompt
│   ├── tool_executor.py                 # NIM tool-call loop, max cfg.llm.max_tool_calls
iterations
│   └── schemas/                         # JSON schemas — one per tool
│       ├── get_eod_data.json
│       ├── get_fundamentals.json
│       ├── get_options_data.json
│       ├── search_news.json
│       ├── get_fii_dii.json
│       ├── place_order.json
│       ├── place_gtt.json
│       ├── check_risk.json
│       └── send_alert.json
│
├── tools/
│   ├── __init__.py                      # TOOL_REGISTRY + TOOL_SCHEMAS
│   ├── market_data.py                   # get_eod_data() — daily OHLCV + all swing
indicators
│   ├── fundamental_data.py              # get_fundamentals() — P/E, EPS growth, debt,
promoter, pledging
│   ├── options_data.py                  # get_options_data() — PCR, IV, max pain, India VIX
│   ├── news_search.py                   # search_news() — Tavily → DDGS fallback
│   ├── fii_dii_data.py                  # get_fii_dii() — daily sector flows from NSE
│   ├── order_execution.py               # place_order() — entry only, reads trading.mode
flag
│   ├── gtt_manager.py                   # place_gtt/modify_gtt/cancel_gtt — reads
trading.mode flag
│   ├── risk_check.py                    # check_risk() — hard gate, confidence-based sizing
```

```
|   └── alerts.py                       # send_alert() + send_approval_request() — Telegram
|
├── risk/
|   ├── __init__.py
|   ├── engine.py                       # SelfHealingRiskEngine — position sizing + circuit
checks
|   ├── position_sizer.py               # Kelly × confidence_score × drawdown_multiplier
|   ├── circuit_breakers.py             # Hard stops: weekly loss, drawdown, max positions
|   └── circuit_limit_checker.py        # NSE circuit filter check for open positions
|
├── data/
|   ├── __init__.py
|   ├── kite_fetcher.py                 # EOD + weekly candle fetch — reads trading.mode
flag
|   ├── nifty200_loader.py              # Downloads + caches Nifty 200 constituent list from
NSE
|   ├── earnings_calendar.py            # Upcoming earnings dates for watchlist stocks
|   ├── corporate_actions.py            # Dividends, bonus, splits — affect position P&L
|   └── indicators/                     # Indicators module — daily timeframe
|       ├── __init__.py                 # Exports calculate_all(candles, cfg)
|       ├── momentum.py                 # RSI(14) · MACD(12/26/9) · Stochastic · ROC — daily
|       ├── trend.py                    # EMA(21/50/200) · ADX(14) · Supertrend — daily
|       ├── volatility.py               # ATR(14) daily · Bollinger Bands — stop placement
|       ├── volume.py                   # OBV · volume ratio · MFI — NO VWAP
|       ├── structure.py                # Weekly pivots · 52-week high proximity · S/R
levels
|       ├── relative_strength.py        # Stock return vs Nifty 50 — 20/50/90 day windows
|       └── patterns.py                 # Daily candlestick patterns — TA-Lib,
cfg.patterns.enabled
|
├── paper/
|   ├── __init__.py
|   ├── fill_engine.py                  # Simulates order fills for paper + backtest modes
|   ├── gtt_simulator.py                # Tracks GTT trigger prices — no real Kite GTT in
paper/backtest
|   └── slippage_model.py               # LTP × 0.001 per side + ₹20 brokerage
|
├── auth/
|   ├── __init__.py
|   ├── token_manager.py                # Daily Kite token refresh at 08:50
|   └── totp_login.py                   # TOTP automation — pyotp + requests
|
├── learning/
|   ├── __init__.py
|   ├── trade_reviewer.py               # Event-driven: runs when each trade closes, logs
observations
|   ├── stats_engine.py                 # Monthly: closed trades → Sharpe, win rate, Kelly
|   ├── lesson_generator.py             # Monthly: NIM reviews trades → SKILL.md.staging
proposals
|   └── skill_updater.py                # Telegram YES/NO → approved lessons → SKILL.md +
git commit
|
├── notifications/
|   ├── __init__.py
|   ├── telegram_client.py              # Outbound: alerts + approval requests
|   └── telegram_handler.py             # Inbound: YES/NO for entries + /pause /resume
/status
|
├── backtest/
|   ├── __init__.py
```

```
|   ├── data_fetcher.py                  # Chunked historical fetch, parquet caching
|   ├── candle_replay.py                 # Daily candle replay — research logic runs each
"day"
|   ├── walk_forward.py                  # In-sample/out-of-sample splits + WFE ratio
|   ├── optimizer.py                     # Optuna Bayesian search over config.yaml params
|   ├── metrics.py                       # QuantStats tearsheet + vectorbt equity curve
|   └── nse_bhav_fetcher.py              # Historical OI/PCR from NSE bhav copy files
|
├── strategy/
|   ├── SKILL.md                         # Trading philosophy — editable asset (v1.0 in
Section 6)
|   ├── SKILL.md.staging                 # Monthly proposals awaiting your approval
|   ├── research_program.md              # Research procedure (v1.0 in Section 6)
|   └── analyst_program.md               # Monthly review procedure
|
├── context/
|   ├── state.json                       # Live: cash, positions, P&L, drawdown,
consecutive_losses
|   ├── trades.json                      # All closed trades — includes skill_version hash
|   ├── stats.json                       # Monthly metrics: Sharpe, win_rate,
Kelly_multiplier
|   ├── pending_approvals.json           # Shortlisted setups awaiting your YES/NO
|   ├── trade_observations.json          # Event-driven notes per closed trade
|   ├── fundamentals_cache.json          # Last-known fundamentals per ticker — layer 4
fallback
|   ├── nifty200.json                    # Cached Nifty 200 constituent list
|   ├── research/                        # Nightly: one JSON per analysed stock
|   |   └── YYYY-MM-DD/{ticker}.json
|   └── daily/
|       └── YYYY-MM-DD.json              # State snapshot for crash recovery
|
├── logs/
|   ├── research.log                     # Every stock scored — full NIM reasoning
|   ├── decisions.log                    # Every order decision
|   ├── trades.log                       # Every order placed, GTT set, GTT triggered
|   └── errors.log                       # Exceptions, API failures, heal events
|
├── reports/
|   └── (auto-created, gitignored — QuantStats HTML + equity curves)
|
└── tests/
    ├── test_risk.py
    ├── test_indicators.py
    ├── test_reconciler.py
    ├── test_paper.py
    ├── test_gtt_simulator.py
    └── test_mode_switching.py           # Verifies flag routing without mocks
```

4. Agents — Deep Dive

4.1 research_agent.py

Runs every evening from 15:45 · The backbone — where all intelligence lives

Property	Detail
Startup	Loads strategy/SKILL.md · strategy/research_program.md · context/state.json (open positions to exclude) · context/stats.json (current Sharpe for scalar metric).
Universe load	data/nifty200_loader.py provides the 200 tickers. Excludes any tickers already held as open positions.
Quick filter (Python)	Before calling NIM: discard if below 200 EMA, market cap < ₹5,000 Cr, avg daily volume < 5L shares, promoter pledging > 30%. This removes ~60% of universe cheaply. No NIM tokens spent.
Per-stock NIM call	For each surviving stock: tool calls in order — get_eod_data(), get_fundamentals(), search_news(), get_fii_dii() (cached), get_options_data() (Nifty 50 stocks only). NIM receives full context + SKILL.md philosophy.
NIM output (JSON)	{score: 0–10, setup_type: breakout|pullback|earnings_play|sector_rotation|skip, entry_zone: {low, high}, stop_price, target_price, holding_days_expected, confidence_reasoning, risk_flags[]}
Scoring gate	Score < 7.0 = discard. Score 7.0–8.0 = shortlist. Score 8.0+ = priority. Max 7 in shortlist. If shortlist + current positions > 3: only shortlist remaining capacity.
Output	context/research/YYYY-MM-DD/{ticker}.json per stock. context/pending_approvals.json = shortlist. logs/research.log = full run.
Morning briefing	Telegram at 08:45: for each shortlisted stock — name, score, setup type, entry zone, stop, target, expected hold, thesis in 2 sentences. Reply YES/NO for each.
Earnings awareness	Checks data/earnings_calendar.py before shortlisting. Avoids entering positions where earnings fall within the expected holding period unless the trade IS an earnings play.
Corporate actions	Checks data/corporate_actions.py. Flags upcoming dividends (stock goes ex-div — price drops), bonus issues, splits. These affect P&L calculations and should be communicated in briefing.
F&O expiry avoidance	Never shortlists entries in the last 2 trading days of F&O expiry week. Volatility and liquidity distortions spike near expiry. Hard rule — not a SKILL.md preference.
Monthly analyst loop	First Sunday of month: if context/stats.json shows >= 8 closed trades this month, runs analyst_program.md process. NIM reviews trades, proposes SKILL.md changes to staging. Sends you Telegram for approval.

4.2 execution_agent.py

Polls every 30 minutes, 09:15–15:30 · Approval handler + GTT lifecycle manager

Property	Detail
Startup (09:15)	1. auth/token_manager refreshes Kite token. 2. reconciler.py runs. 3. Load pending_approvals.json. 4. Check PAUSE file. 5. Send "Execution agent online" Telegram.
Entry approval flow	telegram_handler receives YES → check_risk() → if approved: order_execution.place_order() → on fill: gtt_manager.place_gtt(stop) + gtt_manager.place_gtt(target) → update state.json → confirm Telegram. NO → remove from pending, log.
Approval timeout	Any pending approval not acted on within cfg.execution.approval_timeout_hours (default 16h) is auto-expired. Alert sent: "HDFCBANK setup expired — not entered."
30-min poll	kite.positions() + kite.get_gtts(). For each open position: is the GTT still active? Did price move enough to trail? Did thesis score change overnight? Is a corporate action imminent?
GTT health check	If a GTT disappears without a position close (cancelled by Zerodha, expired): alert immediately. Do not re-place automatically. Require your acknowledgement. Risk is real — don't auto-decide.
Stop trailing	If position unrealised P&L crosses cfg.execution.trail_stop_at_pct (5%): modify_gtt stop to breakeven. At cfg.execution.trail_to_pct (10%): trail stop to entry+5%. Configurable. Only tightens.
Thesis monitoring	Each evening research_agent re-scores open positions against current data. If score drops below 5.0: "RELIANCE thesis weakened (score 4.2). Original: 8.1. Reason: FII selling sector, earnings miss. Consider early exit." Never exits without you.
Circuit limit check	risk/circuit_limit_checker.py runs every 60 min. If a held stock hits NSE circuit limit: alert immediately. GTT SL-M orders cannot fill when circuit is hit — manual intervention required.
PAUSE file	Checks os.path.exists("PAUSE") at every poll. If present: no order placement, no GTT modifications. telegram /pause writes the file. rm PAUSE resumes. Faster than Telegram round-trip.

4.3 reconciler.py

Scenario	Detection	Action
Orphaned position	Kite has position, state.json does not	Alert immediately with position details. Do NOT auto-close. You decide. Bot doesn't know the context.

Stale state	state.json has position, Kite does not	Remove from state.json. Log. GTT probably triggered and state.json wasn't updated (crash scenario).
Missing GTT	Open position exists, stop-loss GTT missing	Alert immediately. Place emergency GTT at original stop price from trades.json.
Full agreement	state.json matches Kite exactly	Log "Reconciliation: OK". Proceed.

5. Tools — What NIM Can Call

Tool	Purpose	Source	Mode-aware?
get_eod_data(ticker)	Daily OHLCV + all 7 indicator categories. First call in every research cycle.	kite.historical_data() OR live EOD	**YES**
get_fundamentals(ticker)	P/E, EPS growth 3yr, debt/equity, promoter holding %, pledging %. Critical for overnight holds — you own this company for 2 weeks.	Screener.in (scrape) + NSE filings	No
get_options_data(ticker)	PCR, max pain, ATM IV, IV percentile, India VIX. Options market confirms or rejects the equity thesis.	Kite Connect	No
search_news(query)	Latest 7 days: earnings, analyst calls, sector events, management commentary, regulatory news.	Tavily → DDGS fallback	No
get_fii_dii()	Daily FII and DII net buy/sell flows by sector. Are foreigners accumulating or selling this sector this week?	NSE public data	No
place_order(...)	Entry limit or market order. Only called after explicit YES from you. Reads trading.mode flag.	kite.place_order() OR paper/fill_engine	**YES**
place_gtt(...)	Places stop-loss GTT and target GTT immediately after entry fill. Two calls per trade. Reads trading.mode flag.	kite.place_gtt() OR paper/gtt_simulator	**YES**
check_risk(...)	Hard gate before every order. Returns approved/rejected + position size based on confidence score.	risk/engine.py	No
send_alert(...)	Telegram push. send_approval_request() variant sends YES/NO buttons for entry decisions.	notifications/telegram_client	No

5.1 paper/gtt_simulator.py — GTT in non-live modes

In paper and backtest modes, real Kite GTT cannot be used (no sandbox exists). gtt_simulator.py maintains an in-memory dict of simulated GTT orders: {position_id: {stop_price, target_price}}. On

every candle in backtest replay, or on every 30-min poll in paper mode, it checks: did the candle's low touch the stop? Did the high touch the target? If yes, simulate a fill via fill_engine.py and close the position. The interface is identical to gtt_manager.py — same function signatures, same return values. The trading.mode flag determines which one gets called. Zero mocks, zero test doubles.

6. Indicators — Daily Timeframe

Seven files in data/indicators/. Same pandas-ta + TA-Lib foundation. All parameters from config.yaml. VWAP removed — it resets daily and is meaningless for multi-day holds. Relative strength added — the most important filter for swing stock selection.

File	Question it answers	Key indicators output
momentum.py	Is this move running out of energy?	rsi_14 · macd · macd_signal · macd_hist · macd_crossover · stoch_k · stoch_d · roc_10
trend.py	Which direction? How strong?	ema_21 · ema_50 · ema_200 · above_200ema · price_vs_ema200_pct · adx · trend_strong · supertrend_direction · supertrend_flipped
volatility.py	How wide should stops be?	atr_14 · atr_pct · stop_distance (ATR × multiplier) · bb_upper · bb_lower · bb_pct · bb_bandwidth · bb_squeeze
volume.py	Is smart money behind this?	obv · obv_trend · volume_ratio · accumulation_flag · mfi
structure.py	Where are the key levels?	weekly_r1 · weekly_r2 · weekly_s1 · weekly_s2 · support · resistance · high_52w · low_52w · proximity_to_52w_high_pct · base_weeks (consolidation duration)
relative_strength.py	Is this stock outperforming?	rs_vs_nifty_20d · rs_vs_nifty_50d · rs_vs_nifty_90d · rs_vs_sector · rs_rank_nifty200 (1=best, 200=worst) · outperforming_index (bool)
patterns.py	What is this daily candle saying?	detected_patterns[] — human-readable list. Only patterns in cfg.indicators.patterns.enabled. Raw TA-Lib integers converted to names. Empty list if none detected.

Key indicator insights for swing trading

- 200 EMA is a hard filter — above it = bull structure, only consider longs. The single most important check.
- relative_strength.py is the second most important filter. A stock lagging its sector in a bull market is broken. Buy only outperformers.
- base_weeks in structure.py counts how many weeks price has been in a tight consolidation range. 4–8 weeks = quality base. Under 2 weeks = too early. Over 12 weeks = may be stale.
- ATR-based stops from volatility.py mean stops automatically widen in high-volatility markets. Config: atr_stop_multiplier: 1.5 — change this to adjust risk without touching code.
- VWAP is deliberately absent. It resets each day and has no meaning for a 10-day hold.

7. The Three Strategy Files — v1.0 Content

These files are more important than any Python code. The Python executes them. They contain the intelligence.

7.1 SKILL.md v1.0

```
# SKILL.md — Swing Trading Philosophy v1.0
# Read by NIM on every stock analysis. Not rules — reasoning patterns.
# Updated monthly by the analyst loop after sufficient trade evidence.

## Identity
I am a swing trading agent for Indian equities (NSE, Nifty 200).
I hold positions 7–14 days. I make 4–8 trades per month.
I am patient. I do not chase. I wait for high-quality setups.

## What I look for — the ideal setup
- A quality company in a strong sector with institutional accumulation visible
- Price consolidating (base building) 4–8 weeks on the weekly chart
- Volume drying up during consolidation — no selling pressure
- Volume expanding on breakout attempts — buyers showing up
- Stock outperforming Nifty 50 on 20 and 50 day basis
- 200 EMA sloping upward — I only swing long in a bull structure
- A catalyst exists or approaches: earnings, contract win, sector tailwind,
  analyst upgrade, policy benefit, management buyback
- Promoter holding stable or increasing, pledging below 20%

## Setup types I trade
- Breakout: price emerging from multi-week base, volume expanding, near or above 52w high
- Pullback to EMA: healthy uptrending stock pulling back to 21/50 EMA on low volume
- Earnings anticipation: strong setup with earnings in 2–4 weeks and rising estimates
- Sector rotation: FII money entering a sector, buy the sector leader

## Immediate disqualifiers — do not shortlist regardless of chart
- Stock below 200 EMA
- Promoter pledging above 30%
- Debt/equity above 2.0 (non-financial companies)
- P/E above 80 without accelerating revenue growth
- Stock has moved more than 30% in last 4 weeks (late entry, chase risk)
- Pending SEBI investigation or serious litigation
- Entry in last 2 days of F&O expiry week

## Position sizing
- Confidence 8.0–10.0: 40% of available capital for this trade
- Confidence 7.0–8.0: 25% of available capital for this trade
- Never hold more than 3 positions simultaneously
- Always maintain 20% cash — dry powder for unexpected opportunities

## Stop-loss and exit
- Stop = entry price minus (1.5 × daily ATR)
- Target = minimum 2× risk (2:1 risk/reward before entering)
- When unrealised gain reaches +5%: move stop to breakeven
- When unrealised gain reaches +10%: trail stop to entry + 5%
- Stops only move in my favour — never widen a stop
```

```
## Lessons learned
(Populated by monthly analyst loop — empty at v1.0)
```

7.2 research_program.md v1.0

```
# research_program.md — Research Procedure v1.0
# Tells the agent HOW to research. Changed only when the process breaks.

## Step 1 — Quick Python filter (no NIM call)
Skip immediately if any condition is true:
- Stock below 200 EMA (from last EOD data)
- Market cap below ₹5,000 Cr
- Average daily volume (20-day) below 5,00,000 shares
- Promoter pledging above 30% (from get_fundamentals cache if available)
- Entry falls in last 2 trading days of current F&O expiry week

## Step 2 — Data fetch (for surviving stocks)
Call tools in this order. Respect rate limits (0.4s between Kite calls).
1. get_eod_data(ticker)          — daily candles + all indicators
2. get_fundamentals(ticker)      — PE, EPS, debt, promoter, pledging
3. search_news(ticker + " stock news India last 7 days")
4. get_fii_dii()                 — cached once per session, not per stock
5. get_options_data(ticker)      — only for Nifty 50 stocks

## Step 3 — NIM analysis
System: this document + SKILL.md
Evaluate the stock through the SKILL.md philosophy.
Required JSON output:
  {
    "score": 0.0-10.0,
    "setup_type": "breakout|pullback|earnings_play|sector_rotation|skip",
    "entry_zone": {"low": float, "high": float},
    "stop_price": float,
    "target_price": float,
    "holding_days_expected": int,
    "confidence_reasoning": "2-3 sentence summary",
    "risk_flags": ["list of specific concerns"]
  }

## Step 4 — Shortlisting rules
- Keep if score >= 7.0
- Maximum 7 stocks in shortlist
- If shortlist would exceed remaining position capacity (3 - open_positions): trim to
capacity
- Sort by score descending when trimming

## Step 5 — Output
- Write full analysis to context/research/YYYY-MM-DD/{ticker}.json
- Write shortlist to context/pending_approvals.json
- Send Telegram briefing at 08:45
```

7.3 analyst_program.md — structure

Not reproduced in full here. It instructs NIM to: (1) load all closed trades from this month, (2) for each losing trade: what condition in SKILL.md was violated or absent? (3) for each winner: what made it qualify? (4) find patterns across all trades, (5) propose at most 3 specific SKILL.md additions with supporting trade evidence. Output must cite specific trade IDs. Vague proposals ("improve momentum filter") are rejected — proposals must be specific ("Add: avoid entries when stock is more than 15% above 50 EMA — RELIANCE trade TRD-001 entered at this condition and immediately reversed").

8. config.yaml — Complete File

Single source of truth. Committed to git. No secrets here — those stay in .env. Every tunable value traces back to this file. No magic numbers anywhere in Python.

```yaml
# swingtradev3/config.yaml
# ALL tunable values live here. Secrets live in .env. Commit this file.

# ── Mode ──────────────────────────────────────────────────
trading:
  mode: paper                       # paper | live  (backtest run via CLI)
  capital_inr: 20000
  exchange: NSE
  universe: nifty200
  max_positions: 3
  min_cash_reserve_pct: 0.20

# ── Research agent ────────────────────────────────────────
research:
  scan_start_time: "15:45"
  briefing_time: "08:45"
  min_score_threshold: 7.0
  max_shortlist: 7
  quick_filter:
    min_market_cap_cr: 5000
    min_avg_volume: 500000
    max_promoter_pledge_pct: 30
    below_200ema_disqualify: true
  max_same_sector_positions: 2      # sector concentration limit
  async_scan: true                  # asyncio.Semaphore(3) for Kite rate limit
  analyst_loop:
    enabled: true
    cadence: monthly                # monthly | quarterly
    day_of_month: 1                 # first Sunday of month
    time: "18:00"
    min_trades_required: 8
  quarterly_audit:
    enabled: true
    min_trades_required: 20

# ── Execution agent ───────────────────────────────────────
execution:
  poll_interval_minutes: 30
  approval_timeout_hours: 16
  trail_stop_at_pct: 5.0
  trail_to_pct: 10.0
  enable_trailing: true
  avoid_fno_expiry_days: 2          # skip entries N days before expiry
  max_entry_deviation_pct: 3.0      # auto-expire if price >3% above entry zone top

# ── Corporate action handling ─────────────────────────────
execution:
  corporate_action_handling:
    dividend_adjust_stop: true
    alert_days_before_exdate: 5
```

```yaml
    auto_adjust_timeout_hours: 12
    bonus_split_pause_entries: true

# ── Risk ──────────────────────────────────────────────────
risk:
  max_risk_pct_per_trade: 0.015
  max_weekly_loss_pct: 0.04
  max_drawdown_pct: 0.10
  min_rr_ratio: 2.0                 # minimum risk:reward before entering
  confidence_sizing:
    high:   {min_score: 8.0, capital_pct: 0.40}
    medium: {min_score: 7.0, capital_pct: 0.25}

# ── Indicators (daily timeframe) ──────────────────────────
indicators:
  timeframe: daily
  candle_buffer_size: 250           # ~1 year of daily candles
  weekly_candle_buffer: 104         # ~2 years of weekly candles

  momentum:
    rsi_length: 14
    rsi_overbought: 70
    rsi_oversold: 30
    macd_fast: 12
    macd_slow: 26
    macd_signal: 9
    stoch_k: 14
    stoch_d: 3
    roc_length: 10

  trend:
    ema_fast: 21
    ema_mid: 50
    ema_slow: 200
    adx_length: 14
    adx_trend_threshold: 25
    supertrend_length: 10
    supertrend_multiplier: 3.0

  volatility:
    atr_length: 14
    atr_stop_multiplier: 1.5
    bb_length: 20
    bb_std: 2.0
    bb_squeeze_threshold: 0.02

  volume:
    volume_avg_periods: 20
    volume_spike_multiplier: 2.0
    mfi_length: 14
    # VWAP: not used for swing trading

  structure:
    pivot_type: weekly
    sr_lookback_periods: 20
    high_52w_proximity_alert_pct: 5.0
```

```yaml
    base_consolidation_min_weeks: 4
    base_consolidation_max_weeks: 12

  relative_strength:
    periods: [20, 50, 90]
    benchmark: "NSE:NIFTY 50"

  patterns:
    min_strength: 60
    enabled:
      - doji
      - hammer
      - engulfing
      - morning_star
      - evening_star
      - shooting_star
      - three_white_soldiers

# ── LLM ──────────────────────────────────────────────────
llm:
  timeout_seconds: 5.0
  max_tool_calls_per_stock: 8
  research_model:
    provider: nim
    model: "deepseek-ai/deepseek-v3-2"
    temperature: 0.15
    max_tokens: 1000
  execution_model:
    provider: nim
    model: "qwen/qwen3.5-122b-a10b"
    temperature: 0.1
    max_tokens: 300
  analyst_model:
    provider: nim
    model: "deepseek-ai/deepseek-v3-2"
    temperature: 0.2
    max_tokens: 2000
  fallback_chain:
    - {provider: groq,      model: "llama-3.3-70b-versatile"}
    - {provider: nim,       model: "stepfun-ai/step-3.5-flash"}
    - {provider: anthropic, model: "claude-sonnet-4-6"}

# ── Learning ─────────────────────────────────────────────
learning:
  min_trades_for_lesson: 8          # monthly minimum
  min_trades_for_kelly: 20          # quarterly minimum
  max_lessons_per_month: 3

# ── Schedule ─────────────────────────────────────────────
schedule:
  auth_refresh: "08:50"
  market_open: "09:15"
  market_close: "15:30"
  research_start: "15:45"
  briefing_time: "08:45"
  timezone: "Asia/Kolkata"
```

```yaml
# ── Notifications ──────────────────────────────────────
notifications:
  telegram:
    enabled: true
    require_entry_approval: true
    alert_on_levels: [warning, critical]
    daily_summary: true
    daily_summary_time: "16:00"

# ── Backtest (run via: python main.py --mode backtest) ──────
backtest:
  use_llm: false                    # false=fast rule-based | true=faithful NIM calls
  start_date: "2023-01-01"
  end_date: "2025-12-31"
  initial_capital: 20000
  fee_model: delivery               # delivery (swing) = 0 brokerage, only STT
  fill_price: next_open             # no look-ahead bias
  slippage_pct: 0.001
  brokerage_per_order: 0            # delivery trades = zero brokerage on Zerodha
  cache_data: true
  cache_dir: ".backtest_cache"
  thresholds:
    min_win_rate: 0.45
    min_sharpe: 1.0
    max_drawdown: 0.15
    min_profit_factor: 1.3
    min_wfe_ratio: 0.5
    min_total_trades: 20
  walk_forward:
    enabled: true
    in_sample_months: 6
    out_sample_months: 2
    n_windows: 4
  optimizer:
    enabled: false
    n_trials: 100
    metric: sharpe
    search_space:
      indicators.momentum.rsi_length:           [10, 21]
      indicators.volatility.atr_stop_multiplier: [1.0, 2.5]
      indicators.trend.adx_trend_threshold:      [20, 30]
      risk.confidence_sizing.medium.capital_pct:  [0.15, 0.35]
```

9. File-by-File Input / Output Specification

9.1 Root

main.py

Module	Root
Purpose	Entry point. Starts research_agent on evening schedule and execution_agent on 30-min market-hours schedule. Accepts --mode backtest CLI flag to run backtest instead.
Inputs	.env via config.py
Outputs	Running scheduled processes
Calls	agents/research_agent · agents/execution_agent · auth/token_manager · backtest/ (if --mode backtest)
Called by	systemd / cron / manual

config.py

Module	Root
Purpose	Loads config.yaml via PyYAML. Validates into Pydantic Settings. Single source of truth for all non-secret config. Exposes a cfg singleton imported by all modules.
Inputs	config.yaml · .env (for trading.mode override)
Outputs	Settings dataclass accessible as cfg everywhere
Calls	PyYAML · pydantic
Called by	Every module

9.2 agents/

agents/research_agent.py

Module	agents/
Purpose	Nightly: loads universe, runs quick filter, fetches data async (asyncio.Semaphore(3) for Kite rate limit), runs NIM analysis, applies sector concentration limit, scores, shortlists, records SKILL.md git hash, writes outputs, sends briefing. Monthly: analyst loop.
Inputs	strategy/SKILL.md · strategy/research_program.md · context/state.json · context/stats.json · data/nifty200_loader · data/earnings_calendar · data/corporate_actions

Outputs	context/research/YYYY-MM-DD/{ticker}.json · context/pending_approvals.json · SKILL.md.staging (monthly) · logs/research.log
Calls	data/kite_fetcher · data/indicators/__init__ · tools/ (all 9) · llm/tool_executor · learning/lesson_generator (monthly) · gitpython (skill_version hash)
Called by	main.py (scheduled 15:45)

agents/execution_agent.py

Module	agents/
Purpose	Polls every 30 min. Handles approval flow with entry validity check. Places entry orders and GTTs. Monitors positions. Checks corporate actions and adjusts GTTs. Trails stops. Runs reconciler at startup.
Inputs	context/pending_approvals.json · context/state.json · config.yaml · PAUSE file
Outputs	Orders in Zerodha (or paper/fill_engine) · GTTs in Zerodha (or paper/gtt_simulator) · Updated state.json · logs/trades.log · Telegram alerts
Calls	agents/reconciler · tools/order_execution · tools/gtt_manager · tools/risk_check · tools/alerts · risk/circuit_limit_checker · data/corporate_actions
Called by	main.py (scheduled 09:15–15:30)

agents/reconciler.py

Module	agents/
Purpose	Startup check. Compares state.json vs live Kite. Four scenarios: orphaned, stale, missing GTT, full agreement.
Inputs	context/state.json · kite.positions() · kite.get_gtts()
Outputs	Corrected state.json · Telegram alerts if mismatch · Emergency GTT if stop missing
Calls	kiteconnect · tools/gtt_manager · notifications/telegram_client
Called by	agents/execution_agent (09:15 startup)

9.3 llm/

llm/nim_client.py

Module	llm/
Purpose	Primary LLM caller. OpenAI SDK pointed at NIM base URL.
Inputs	messages: list[dict] · tools: list[dict] · model: str
Outputs	Raw OpenAI-compatible response

Calls	openai.OpenAI · llm/schemas/*.json
Called by	llm/router.py

llm/router.py

Module	llm/
Purpose	Fallback chain with asyncio.wait_for timeout. NIM (5s) → Groq → Gemini Flash → Claude. Logs provider used every call.
Inputs	messages, tools, model hint · cfg.llm.timeout_seconds · cfg.llm.fallback_chain
Outputs	Response from first provider to succeed · provider_used: str
Calls	llm/nim_client · openai SDK (Groq/Gemini) · anthropic SDK (Claude)
Called by	llm/tool_executor

llm/prompt_builder.py

Module	llm/
Purpose	Assembles system prompt for every NIM call. SKILL.md + research_program.md + portfolio state + stats. Target ~1,500 tokens.
Inputs	strategy/SKILL.md · strategy/research_program.md · context/state.json · context/stats.json · stock-specific context
Outputs	system_prompt: str
Calls	File reads only
Called by	llm/tool_executor

llm/tool_executor.py

Module	llm/
Purpose	NIM tool-call loop. Dispatches tool calls to TOOL_REGISTRY. Loops until final text response. Max cfg.llm.max_tool_calls_per_stock.
Inputs	system_prompt · stock context · TOOL_REGISTRY from tools/__init__
Outputs	Final decision dict: {score, setup_type, entry_zone, stop_price, target_price, holding_days_expected, confidence_reasoning, risk_flags}
Calls	llm/router · tools/__init__
Called by	agents/research_agent · learning/lesson_generator

9.4 tools/

tools/market_data.py

Module	tools/
Purpose	Fetches EOD daily candles and runs calculate_all() indicators. First tool called per stock in research cycle. Reads trading.mode flag for data source.
Inputs	ticker: str · cfg.indicators
Outputs	{price, change_pct, + all indicator outputs from all 7 indicator modules}
Calls	data/kite_fetcher · data/indicators/__init__
Called by	llm/tool_executor via TOOL_REGISTRY

tools/fundamental_data.py

Module	tools/
Purpose	4-layer fundamentals fetcher. Layer 1: yfinance (PE, EPS, debt/equity, sector). Layer 2: nsepython + nsetools (promoter holding, pledging — best-effort). Layer 3: Firecrawl /extract fallback. Layer 4: cache (never errors). See Section 14.1 for full detail.
Inputs	ticker: str
Outputs	{pe_ratio, eps_growth_3yr_pct, debt_equity, promoter_holding_pct, promoter_pledge_pct, revenue_growth_pct, roce, sector, is_stale: bool}
Calls	yfinance · nsepython · nsetools · firecrawl-py · context/fundamentals_cache.json
Called by	llm/tool_executor via TOOL_REGISTRY

tools/fii_dii_data.py

Module	tools/
Purpose	Daily FII/DII net flows by segment and sector. Cached once per session.
Inputs	No inputs — fetches today's data from NSE
Outputs	{date, fii_net_crore, dii_net_crore, sector_flows: {sector: fii_net}}
Calls	requests · pandas · NSE public data
Called by	llm/tool_executor · agents/research_agent (caches result)

tools/gtt_manager.py

Module	tools/
Purpose	GTT lifecycle management. Reads trading.mode flag — calls kite.place_gtt() in live mode, paper/gtt_simulator.py in paper/backtest.

Inputs	ticker, transaction_type, trigger_price, limit_price, qty · cfg.trading.mode
Outputs	{gtt_id, status, trigger_price} · Updates state.json with gtt_ids
Calls	kiteconnect.place_gtt / modify_gtt / cancel_gtt OR paper/gtt_simulator
Called by	agents/execution_agent · agents/reconciler

tools/order_execution.py

Module	tools/
Purpose	Entry orders only. Reads trading.mode flag. Calls check_risk first. On live fill: calls gtt_manager for stop + target GTTs immediately.
Inputs	ticker, side, qty, order_type, price · cfg.trading.mode
Outputs	{order_id, status, average_price, stop_gtt_id, target_gtt_id}
Calls	tools/risk_check · kiteconnect.place_order OR paper/fill_engine · tools/gtt_manager
Called by	agents/execution_agent

9.5 paper/

paper/fill_engine.py

Module	paper/
Purpose	Used in paper and backtest modes. Intercepts order calls. Fills at next-day open + slippage. Updates in-memory portfolio state identically to live.
Inputs	Signal dict {ticker, side, qty, order_type} · next_candle open price · cfg.backtest.slippage_pct
Outputs	Fill result {filled_price, qty_filled, brokerage, net_pnl} matching live format
Calls	paper/slippage_model
Called by	tools/order_execution (when mode != live)

paper/gtt_simulator.py

Module	paper/
Purpose	Simulates GTT orders in paper and backtest modes. Maintains in-memory {position_id: {stop_price, target_price}}. Checked on every 30-min poll (paper) or every candle (backtest).
Inputs	GTT placement calls from gtt_manager · Daily candle OHLCV for trigger checking

Outputs	Simulated GTT trigger events matching live kite.get_gtts() response format
Calls	paper/fill_engine (on trigger)
Called by	tools/gtt_manager (when mode != live) · backtest/candle_replay

9.6 data/

data/kite_fetcher.py

Module	data/
Purpose	Fetches daily and weekly candles. Reads trading.mode: backtest = chunked historical with parquet cache, paper|live = latest EOD. Rate-limited with 0.4s sleep between calls.
Inputs	ticker: str · interval: day|week · cfg.trading.mode · cfg.backtest dates
Outputs	pd.DataFrame with columns [open, high, low, close, volume, date] · Parquet cache in .backtest_cache/
Calls	kiteconnect · pandas · pyarrow
Called by	tools/market_data · backtest/data_fetcher · agents/research_agent

data/nifty200_loader.py

Module	data/
Purpose	Maintains Nifty 200 constituent list. Downloads from NSE weekly and caches. Provides the scan universe to research_agent.
Inputs	NSE index constituents page
Outputs	List[str] — 200 ticker symbols · Cached to context/nifty200.json
Calls	requests · pandas
Called by	agents/research_agent

data/earnings_calendar.py

Module	data/
Purpose	Fetches upcoming earnings/results dates for all Nifty 200 stocks. Research agent checks this before shortlisting to avoid accidental earnings exposure.
Inputs	NSE corporate calendar endpoint
Outputs	Dict {ticker: next_results_date} for next 30 days
Calls	requests · pandas

Called by	agents/research_agent

data/corporate_actions.py

Module	data/
Purpose	Tracks upcoming dividends, bonus issues, splits, rights for held positions. Ex-dividend dates cause price drops — position P&L needs adjustment.
Inputs	NSE corporate actions endpoint · context/state.json (held positions)
Outputs	List[CorporateAction] — {ticker, action_type, date, value}
Calls	requests · pandas
Called by	agents/execution_agent (daily monitoring) · agents/research_agent (briefing flag)

9.7 learning/

learning/trade_reviewer.py

Module	learning/
Purpose	Event-driven — fires when each trade closes. Logs structured observation: did thesis hold? what triggered exit? how did NIM reasoning compare to reality? No SKILL.md changes — observation only.
Inputs	Closed trade dict from state.json · Original research context from context/research/
Outputs	context/trade_observations.json — appended observation per trade · logs/research.log
Calls	File reads only
Called by	agents/execution_agent (on GTT trigger / position close)

learning/stats_engine.py

Module	learning/
Purpose	Monthly: recalculates performance metrics from trades.json. Respects min_trades_for_kelly threshold.
Inputs	context/trades.json
Outputs	context/stats.json: {win_rate, sharpe, avg_winner_pct, avg_loser_pct, kelly_multiplier, best_setup_type, worst_setup_type}
Calls	pandas · numpy
Called by	main.py (monthly cron) · agents/research_agent (monthly loop)

learning/lesson_generator.py

Module	learning/
Purpose	Monthly: NIM analyses closed trades + trade_observations.json + current SKILL.md. Proposes max 3 specific edits backed by trade evidence.
Inputs	context/trades.json (this month) · context/trade_observations.json · strategy/SKILL.md · strategy/analyst_program.md · cfg.learning
Outputs	strategy/SKILL.md.staging — proposed additions with supporting trade IDs
Calls	llm/router (DeepSeek V3.2 analyst model)
Called by	main.py (monthly cron) · agents/research_agent (monthly loop)

learning/skill_updater.py

Module	learning/
Purpose	Telegram YES/NO approval flow. Approved lessons appended to SKILL.md and git-committed.
Inputs	strategy/SKILL.md.staging · Telegram reply
Outputs	Updated strategy/SKILL.md · git commit "Monthly lesson YYYY-MM"
Calls	notifications/telegram_client · gitpython
Called by	notifications/telegram_handler (on YES reply)

9.8 backtest/

backtest/candle_replay.py

Module	backtest/
Purpose	Replays daily candles chronologically. For each "trading day": runs research logic (which stocks qualify today?), simulates entries via fill_engine, checks GTT triggers via gtt_simulator, updates in-memory portfolio.
Inputs	Dict {ticker: pd.DataFrame} from data_fetcher · cfg.backtest · cfg.research
Outputs	List[BacktestTrade] — full trade history with entry, exit, P&L, reasoning
Calls	data/indicators/__init__ · paper/fill_engine · paper/gtt_simulator · risk/engine · data/earnings_calendar
Called by	backtest/walk_forward · backtest/optimizer · main.py (--mode backtest)

backtest/metrics.py

Module	backtest/
Purpose	Generates QuantStats HTML tearsheet and vectorbt equity curve. Checks all go/no-go thresholds. Returns pass/fail verdict.
Inputs	List[BacktestTrade] · cfg.backtest.thresholds · Nifty 50 as benchmark
Outputs	MetricsReport {sharpe, win_rate, profit_factor, max_drawdown, passed: bool} · HTML reports in reports/
Calls	quantstats · vectorbt · plotly
Called by	backtest/walk_forward · backtest/optimizer · main.py

10. APIs, Keys, and Monthly Cost

Service	Required	Cost	Where to get it
Zerodha account	**YES**	₹200 one-time	zerodha.com → Open Account → complete KYC
Kite Connect API	**YES**	₹500/month	kite.trade → My Apps → Create App → get API key + secret
NVIDIA NIM	**YES**	**FREE**	build.nvidia.com → Sign in with Google → key auto-generated on dashboard
Tavily Search	**YES**	**FREE (1K/mo)**	tavily.com → Sign up → Dashboard → API Keys
Telegram Bot	**YES**	**FREE**	@BotFather → /newbot · @userinfobot for your chat ID
DuckDuckGo DDGS	**YES**	**FREE**	pip install duckduckgo-search — no signup needed
Groq API	**Recommended**	**FREE**	console.groq.com → Sign up → API Keys → Create
Claude API	Optional	~₹30/mo	console.anthropic.com → API Keys (black swan fallback only)

Item	Monthly cost	Note
Kite Connect API	**₹500**	Only recurring cost. Delivery trades = ₹0 brokerage on Zerodha.
NIM + Groq + Gemini + Telegram + Tavily + DDGS	**₹0**	All LLM inference, news, search, alerts — free.
Claude API (black swan fallback)	~₹30	~5 calls/month on genuinely unusual macro events.
TOTAL	**₹530**	Plus STT on delivery trades (~0.1% of sell side).

11. Python Dependencies (requirements.txt)

Package	Version	Module	Purpose
kiteconnect	>=4.1.0	data/ tools/ auth/	Kite historical data, place_order, place_gtt, positions
openai	>=1.30.0	llm/	OpenAI-compatible SDK — NIM, Groq, Gemini
anthropic	>=0.26.0	llm/router.py	Claude fallback — black swan only
tavily-python	>=0.3.0	tools/news_search.py	AI-native news search
duckduckgo-search	>=5.0.0	tools/news_search.py	Free news fallback, no key
python-telegram-bot	>=21.0	notifications/	Alerts + YES/NO entry approval
pandas	>=2.0.0	data/ learning/	Data manipulation foundation
numpy	>=1.26.0	data/ risk/	Numerical calculations
pandas-ta	>=0.3.14b	data/indicators/	Primary — 150+ indicators as df.ta.method()
TA-Lib	>=0.4.28	data/indicators/patterns	60 candlestick patterns — C library, pandas-ta uses automatically
vectorbt	>=0.25.0	backtest/	Portfolio simulation engine
quantstats	>=0.0.62	backtest/metrics.py	HTML tearsheet — Sharpe, Sortino, drawdown
optuna	>=3.6.0	backtest/optimizer.py	Bayesian parameter optimisation
plotly	>=5.20.0	backtest/metrics.py	Interactive equity curves
pyarrow	>=15.0.0	backtest/ data/	Parquet caching for historical candles
numba	>=0.59.0	dependency	JIT — used internally by vectorbt + pandas-ta
tqdm	>=4.66.0	backtest/ data/	Progress bars for long runs
pyotp	>=2.9.0	auth/totp_login.py	TOTP 6-digit code for Kite daily auth
pydantic	>=2.5.0	config.py	Typed config validation
PyYAML	>=6.0.0	config.py	Load config.yaml
tenacity	>=8.2.0	llm/ tools/	Retry with exponential backoff
aiohttp	>=3.9.0	tools/	Async HTTP
python-dotenv	>=1.0.0	config.py	Load .env secrets

schedule	>=1.2.0	main.py	Cron-style scheduler
loguru	>=0.7.0	logs/	Structured logging
gitpython	>=3.1.0	learning/skill_updater	Auto-commit SKILL.md changes
yfinance	>=0.2.40	tools/fundamental_data	Fundamentals layer 1: PE, EPS, debt/equity, sector. Free, no key.
nsepython	>=2.9	tools/fundamental_data	Fundamentals layer 2: promoter holding, pledging. Unofficial NSE wrapper.
nsetools	>=0.0.7	tools/fundamental_data	Fundamentals layer 2 supplement: NSE quotes, book value. Unofficial.
firecrawl-py	>=1.0.0	tools/fundamental_data	Fundamentals layer 3 fallback: AI extraction from Screener.in. Needs FIRECRAWL_API_KEY.
beautifulsoup4	>=4.12.0	tools/fii_dii_data.py	Parse NSE FII/DII HTML tables — not used for fundamentals, replaced by yfinance/nsepython
requests	>=2.31.0	tools/ data/	HTTP calls — NSE, Screener, fundamentals
pytest	>=8.0.0	tests/	Unit tests
pytest-asyncio	>=0.23.0	tests/	Async test support

12. Environment Variables (.env.example)

Secrets only. Every number, flag, and tunable value lives in config.yaml — not here.

```
# ── ZERODHA ──────────────────────────────────────────────
KITE_API_KEY=
KITE_API_SECRET=
KITE_USER_ID=
KITE_PASSWORD=
KITE_TOTP_SECRET=                   # 16-char base32 from Kite 2FA setup
KITE_ACCESS_TOKEN=                  # Written daily by auth/token_manager.py
KITE_REDIRECT_URL=http://localhost:5000

# ── NVIDIA NIM ───────────────────────────────────────────
NIM_API_KEY=                        # Format: nvapi-xxxxxxxx
NIM_BASE_URL=https://integrate.api.nvidia.com/v1

# ── SEARCH ───────────────────────────────────────────────
TAVILY_API_KEY=

# ── TELEGRAM ─────────────────────────────────────────────
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=                   # From @userinfobot

# ── FALLBACK LLMs ────────────────────────────────────────
GROQ_API_KEY=
GROQ_BASE_URL=https://api.groq.com/openai/v1
ANTHROPIC_API_KEY=                  # Optional — black swan fallback

# NOTE: All trading parameters live in config.yaml, not here.
# This file is SECRETS ONLY.
```

13. Launch Checklist

Step 1 — Accounts and API keys

- Open Zerodha account + complete KYC — zerodha.com
- Enable TOTP 2FA on Zerodha — required for auth/ automation
- Subscribe to Kite Connect — kite.trade — ₹500/month — create app, copy API key + secret
- Sign in to build.nvidia.com — NIM key auto-generated on dashboard
- Sign up at tavily.com — generate free API key
- Create Telegram bot via @BotFather — copy token + get chat ID from @userinfobot
- Sign up at console.groq.com — create API key

Step 2 — Repository setup

- mkdir swingtradev3 && git init
- Create all module folders with __init__.py
- Copy config.yaml from Section 8. trading.mode: paper to start.
- Create .env with your real keys — add .env to .gitignore immediately
- Commit config.yaml and .env.example — never commit .env
- Write strategy/SKILL.md from Section 7 starting content
- Write strategy/research_program.md from Section 7 starting content
- Write strategy/analyst_program.md — see Section 7.3 for structure

Step 3 — Build order

- config.py — everything depends on this. Build and test first.
- auth/ — token_manager + totp_login. Test the full auth flow before anything else.
- data/ — kite_fetcher, nifty200_loader, earnings_calendar, corporate_actions, indicators/
- paper/ — fill_engine, gtt_simulator, slippage_model. Test against known candle data.
- tools/ — all 9 tools. Test each with real Kite data + paper mode.
- risk/ — engine, position_sizer, circuit_breakers, circuit_limit_checker
- llm/ — schemas first, nim_client, router, prompt_builder, tool_executor
- agents/ — reconciler, then research_agent, then execution_agent
- learning/ — trade_reviewer, stats_engine, lesson_generator, skill_updater
- notifications/ — telegram_client, telegram_handler
- backtest/ — data_fetcher, candle_replay, walk_forward, optimizer, metrics, nse_bhav_fetcher
- tests/ — write alongside each module, not after
- main.py — final wiring

Step 4 — Backtest SKILL.md v1.0 before any trading

- python main.py --mode backtest
- Fetch 2 years of Nifty 200 daily candles (this takes ~10 min first run, then cached)
- Replay through research logic with backtest.use_llm: false (fast mode)
- All thresholds in config.yaml backtest.thresholds must pass
- Walk-forward WFE ratio must be above 0.5
- If any threshold fails: edit SKILL.md, re-run. Do not skip this step.
- Once passing: run one more backtest with backtest.use_llm: true for final validation (slow, but tests actual NIM reasoning)

Step 5 — Paper trade (minimum 6 weeks, 15+ trades)

- config.yaml trading.mode: paper
- Run every day as if real money. Reply YES/NO to every Telegram briefing.
- Review research.log every evening. Understand every NIM score and reasoning.
- Let the first monthly analyst loop run — approve lessons, watch SKILL.md evolve.
- Only proceed to live after 15+ closed paper trades and positive Sharpe

Step 6 — Go live

- config.yaml trading.mode: live
- ₹20,000 capital. This is a validation amount. Treat it as an extended paper trade.
- First month live: review every Telegram alert the same day
- Check trades.log weekly — how are real fills vs paper fills comparing?
- Scale capital only after 3 months live with consistent positive Sharpe

Disclaimer

Automated trading is permitted for personal Indian accounts under SEBI regulations. Swing trading holds positions overnight through earnings, RBI meetings, and geopolitical events. ₹20,000 is small — you can lose all of it. This document is a technical specification, not financial advice.

14. v6 Design Additions

Six additions incorporated in v6. Each is a targeted improvement — no architecture changes.

14.1 get_fundamentals() — 4-layer architecture

Replaces the single fragile Screener.in BeautifulSoup scraper with a layered approach. Each layer handles what the layer above it cannot. Never errors out — always returns something.

Layer	Source	Fields covered	Reliability
1	yfinance	PE ratio, EPS, debt/equity, market cap, dividend yield, sector, industry. pip install yfinance — no API key.	High — stable, well-maintained
2	nsepython + nsetools	Promoter holding %, promoter pledging %, FII/DII institutional holding. Indian-specific data unavailable elsewhere.	Medium — unofficial NSE wrappers. Has been blocked before. Best-effort only.
3	Firecrawl /extract	Any field layers 1+2 failed to return. AI-based extraction handles HTML changes without selector maintenance.	High when called — but paid. Use minimally. Free tier: 500 credits/month.
4	context/fundamentals_cache.json	Last-known values for any field. Returns with is_stale: true flag. NIM is told data age. Never errors.	Always works — pure file read

nsepython / nsetools fragility — important note

Both nsepython and nsetools wrap NSE's internal undocumented endpoints that were never designed for public consumption. NSE has blocked these libraries before without warning. They are layer 2 — not layer 1 — for exactly this reason. Use them only for the fields (promoter holding, pledging) that yfinance cannot provide. Always wrap in try/except with fallback to the cache. Never depend on them for critical path execution.

```
# tools/fundamental_data.py — 4-layer architecture

Layer 1 — yfinance (primary, reliable)
  Returns: pe_ratio, eps, debt_equity, market_cap, dividend_yield, sector
  API: yf.Ticker(ticker+".NS").info
  Reliability: high — Yahoo Finance is stable, well-maintained
  Limitation: no promoter holding, no pledging data

Layer 2 — nsepython + nsetools (Indian-specific)
  Returns: promoter_holding_pct, promoter_pledge_pct, FII holding, DII holding
  API: nsepython.nse_eq() + nsetools.get_quote()
  Reliability: MEDIUM — unofficial NSE API wrappers. NSE has blocked these
             before without notice. Use only for fields unavailable elsewhere.
  Fragility note: these wrap NSE internal endpoints not designed for public use.
                  Treat as best-effort, not guaranteed.

Layer 3 — Firecrawl /extract (fallback, paid)
  Returns: any field that layers 1+2 failed to provide
```

```
    API: firecrawl.extract(url="https://screener.in/company/{ticker}/",
                           schema=FundamentalsSchema)
    When used: only when layers 1+2 both fail for a specific field
    Cost: minimal — Firecrawl uses AI extraction, handles HTML changes
          Schema-based: describe the field, not CSS selectors.

  Layer 4 — cache (last resort)
    Returns: last-known values from context/fundamentals_cache.json
    When used: when all three above fail
    Behaviour: returns stale data with a is_stale: true flag
    NIM is told: "fundamentals data is stale (last updated N days ago)"
    Never errors out — always returns something
```

New dependencies to add to requirements.txt:

Package	Version	Purpose
yfinance	>=0.2.40	Layer 1: PE, EPS, debt/equity, sector via Yahoo Finance. No API key. Free.
nsepython	>=2.9	Layer 2: Promoter holding, pledging from NSE. No API key. Treat as best-effort.
nsetools	>=0.0.7	Layer 2 supplement: NSE quotes, book value, 52w highs. No API key.
firecrawl-py	>=1.0.0	Layer 3 fallback: AI extraction from Screener.in. Free tier 500 credits/month. Needs FIRECRAWL_API_KEY in .env.

New .env entry: FIRECRAWL_API_KEY= (optional — only needed when layers 1+2 miss fields)

14.2 GTT adjustment logic for corporate actions

Zerodha GTT orders do not adjust for corporate actions. An ex-dividend price drop can trigger your stop-loss even though the thesis is intact. This logic runs every evening during position monitoring.

Action type	Price effect	Bot action	Human action required?
Dividend	Price drops by dividend amount on ex-date	Alert 5 days before. Propose adjusted stop = current_stop - dividend_amount. Auto-applies after 12h if no reply.	Optional — reply NO to keep original stop
Bonus / Split	Price halves (2:1 bonus) or adjusts by split ratio	Alert immediately. Pause new entries for this ticker. Zerodha cancels GTT automatically — reconciler detects and alerts to re-place at new price.	**YES — manual re-entry of GTT after bonus/split**

Rights issue	Minimal spot price effect at announcement	Informational alert only. No GTT action.	Awareness only

```
# execution_agent.py + data/corporate_actions.py — GTT adjustment logic

## Every evening during position poll:

for each open position:
  actions = corporate_actions.get_upcoming(ticker, days=5)

  if action.type == "dividend":
    adjusted_stop = current_stop - action.dividend_amount
    if not state.pending_corporate_action.gtt_adjustment_sent:
      send_alert("RELIANCE ex-div ₹15 on Thursday. Stop will adjust
                  from ₹1,430 → ₹1,415. Reply NO within 12h to keep original.")
      state.pending_corporate_action.gtt_adjustment_sent = True
    elif hours_since_alert > 12 and no_reply_received:
      gtt_manager.modify_gtt(stop_gtt_id, new_trigger=adjusted_stop)
      log("GTT stop auto-adjusted for dividend: ₹1,430 → ₹1,415")

  if action.type in ["bonus", "split"]:
    send_alert("RELIANCE 2:1 bonus on Friday. Zerodha will cancel your GTT.
                Price will halve. Manual review required before ex-date.")
    state.pending_corporate_action.requires_manual_action = True
    # Do NOT auto-adjust — price change is too large, thesis may change
    # After bonus: reconciler.py will detect missing GTT and alert to re-place

  if action.type == "rights":
    send_alert("RELIANCE rights issue. No GTT action needed, for awareness only.")

## New state.json field per position:
"pending_corporate_action": {
  "type": "dividend",
  "amount": 15.0,
  "ex_date": "2026-03-25",
  "gtt_adjustment_sent": false,
  "requires_manual_action": false
}
```

New config.yaml block:

```
execution:
  corporate_action_handling:
    dividend_adjust_stop: true        # propose stop adjustment for dividends
    alert_days_before_exdate: 5       # warn 5 days before ex-date
    auto_adjust_timeout_hours: 12     # auto-apply if no reply within 12h
    bonus_split_pause_entries: true   # pause new entries near bonus/split
```

14.3 Model-agnostic role-based LLM config

Roles are stable. Models filling them change any time without touching Python code. No model name ever appears in a Python file — only in config.yaml. The design has always been architecturally model-agnostic; this section makes it the explicit documented rule.

Role	Default model	Purpose	Swap criteria
research	deepseek-ai/deepseek-v3-2	Nightly stock analysis — deep reasoning, long context, 4–8 tool calls per stock	Swap if: response quality drops, rate limits worsen, better model released
execution	qwen/qwen3.5-122b-a10b	Entry/exit decisions — fast JSON response, <300 tokens, sub-2s target	Swap if: latency too high at market open
analyst	deepseek-ai/deepseek-v3-2	Monthly SKILL.md review — nuanced trade analysis, long context	Swap if: lesson quality is poor after 2 monthly cycles

Rule: Python code always reads cfg.llm.roles.research.model — never a hardcoded string like "deepseek-v3-2". If this rule is ever violated, the model-agnostic architecture breaks.

14.4 SKILL.md version tagging on every trade

Every trade in trades.json records which git commit hash of SKILL.md was active when NIM analysed the stock. This makes SKILL.md evolution traceable — you can precisely measure whether a change improved or hurt performance, and revert to any previous version instantly.

```
# Every trade entry in context/trades.json includes:
"skill_version": "a3f2c1d",   # git commit hash of SKILL.md at time of analysis
"research_date": "2026-03-16",

# How it is set in research_agent.py:
import git
repo = git.Repo(".")
skill_hash = repo.head.commit.hexsha[:7]  # short hash e.g. "a3f2c1d"

# Why this matters:
# Monthly analyst loop can now ask:
# "Trades with skill_version a3f2c1d had Sharpe 1.4"
# "Trades with skill_version b9e1f3a had Sharpe 0.6"
# "What changed between these two versions?" → git diff a3f2c1d b9e1f3a
# This makes SKILL.md evolution traceable and reversible.
```

New field in context/trades.json: "skill_version": "a3f2c1d" — added by research_agent.py at time of analysis using gitpython (already in dependencies).

New query the monthly analyst can now answer: "What was the rolling Sharpe on SKILL.md versions before and after the edit on 2026-04-01?" — directly attributable performance improvement measurement.

14.5 Sector concentration check

Three concurrent positions all in Banking is three correlated bets, not three independent ones. An RBI surprise or sector-specific event moves all three against you simultaneously. The shortlisting step now enforces a sector cap before sending the briefing.

```
# agents/research_agent.py — sector concentration check

# After scoring all stocks, before finalising shortlist:
def apply_sector_concentration_limit(shortlist, open_positions, cfg):
    sector_counts = count_sectors(open_positions)  # from yfinance .info["sector"]
    filtered = []
    for stock in shortlist:
        sector = stock["sector"]
        current_count = sector_counts.get(sector, 0)
        if current_count >= cfg.research.max_same_sector_positions:
            log(f"Skipping {stock['ticker']} — {sector} sector already at limit")
            continue
        sector_counts[sector] = current_count + 1
        filtered.append(stock)
    return filtered

# config.yaml:
# research:
#   max_same_sector_positions: 2
```

New config.yaml value: research.max_same_sector_positions: 2. Sectors sourced from yfinance .info["sector"] — available for free from layer 1 of get_fundamentals().

Example scenario	Without sector cap	With sector cap
Open: HDFCBANK (Banking). Shortlist: ICICIBANK (Banking, score 8.1), INFY (IT, score 7.8), KOTAKBANK (Banking, score 7.5)	All three shortlisted. User approves HDFCBANK + ICICIBANK + KOTAKBANK. Three banking positions. RBI hikes rates → all three drop.	ICICIBANK shortlisted (Banking count now 2). INFY shortlisted. KOTAKBANK skipped (Banking already at limit). Diversified exposure.

14.6 Entry price validity window

You approve a trade at 08:45 based on an entry zone of ₹1,640–1,660. By 11:30 AM when execution_agent polls, RELIANCE has moved to ₹1,720 — 4% above the zone. Entering now is chasing a breakout that already happened. This approval should auto-expire.

```
# execution_agent.py — entry price validity check on every approval
```

```
def is_entry_still_valid(approval, current_price, cfg):
    entry_high = approval["entry_zone"]["high"]
    max_allowed = entry_high * (1 + cfg.execution.max_entry_deviation_pct / 100)
    if current_price > max_allowed:
        send_alert(
            f"{approval['ticker']} approval expired: current ₹{current_price} is",
            f"{((current_price/entry_high)-1)*100:.1f}% above entry zone.",
            f"Entering now = chasing. Auto-cancelled."
        )
        remove_from_pending_approvals(approval)
        return False
    return True

# config.yaml:
# execution:
#   max_entry_deviation_pct: 3.0   # auto-expire if price >3% above entry zone top
```

New config.yaml value: execution.max_entry_deviation_pct: 3.0. When current price exceeds the entry zone top by this percentage, the approval is automatically cancelled with a Telegram alert explaining why. The stock may re-appear in tomorrow's scan if it pulls back into the zone.

14.7 Async scan in research_agent

Sequential scan: 80 stocks × 4 calls × 3s = ~16 minutes. With asyncio.gather and a rate-limit semaphore, this drops to 2–5 minutes. The architecture already uses asyncio throughout. This is a documented implementation pattern, not an architecture change.

```
# agents/research_agent.py — async scan pattern

# Rate limit: Kite allows 3 requests/sec
semaphore = asyncio.Semaphore(3)

async def analyse_stock(ticker):
    async with semaphore:
        data = await get_eod_data(ticker)
        fundamentals = await get_fundamentals(ticker)
        news = await search_news(ticker)
        # NIM call is not rate-limited by Kite — runs freely
        return await nim_score(ticker, data, fundamentals, news)

# Run 10 stocks concurrently instead of sequentially
results = await asyncio.gather(
    *[analyse_stock(t) for t in surviving_tickers]
)

# Sequential:  80 stocks × 3s = 240s (~4 min)
# Concurrent:  80 stocks ÷ 10 parallel × 3s = 24s (~30s)
# Real-world:  NIM latency varies — expect 2-5 minutes total
```

Approach	Time for 80 stocks	Constraint

Sequential (current v5)	16–45 minutes depending on NIM latency	Simple to implement, easy to debug
Async with Semaphore(3) (v6)	2–8 minutes typical	asyncio.Semaphore(3) respects Kite 3 req/sec limit. NIM calls unbounded — parallel fine.

Note: NIM calls are not Kite calls — they are not subject to the 3 req/sec limit. asyncio.Semaphore(3) controls only the Kite data fetch calls. Multiple NIM calls can run in parallel without restriction.